

Как работает автодополнение (поисковая подсказка, suggest)?

Автодополнение (поисковая подсказка, suggest) — это технология, которая в режиме реального времени предлагает пользователю возможные варианты завершения поискового запроса по мере ввода первых символов.

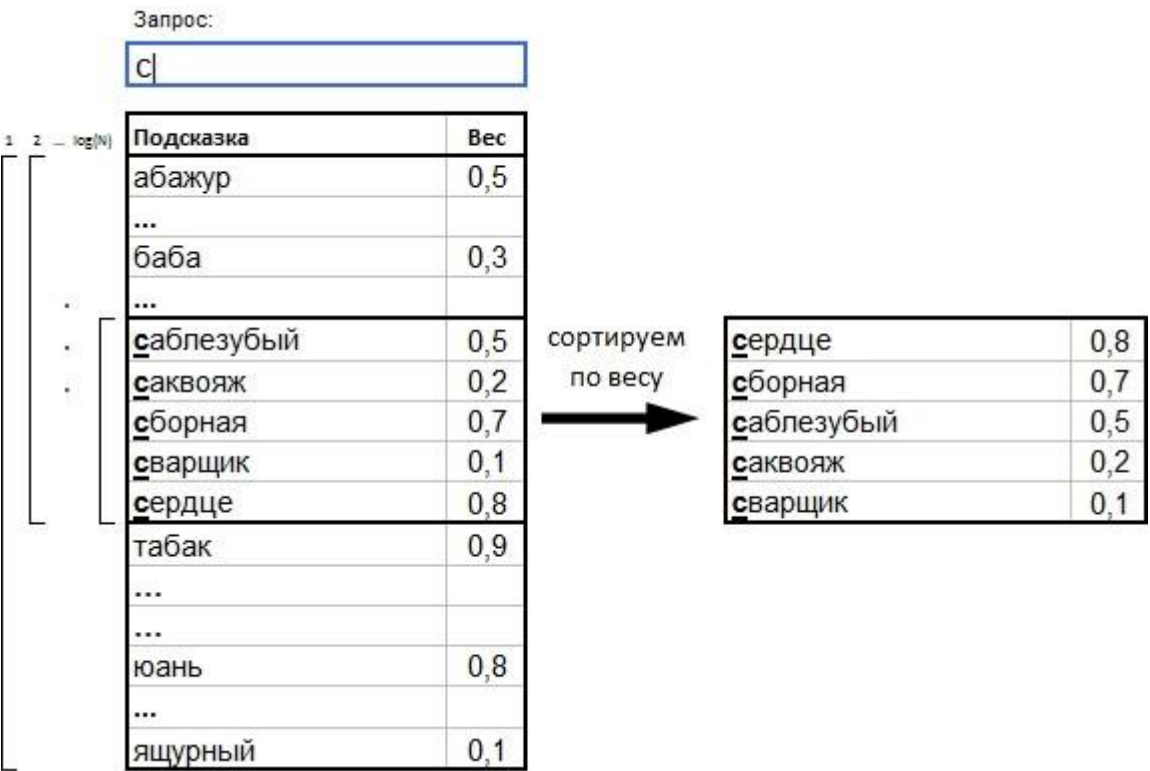
Требования к системе поиска подсказок:

- 1. Полнотекстовый поиск: Система должна находить подсказки, содержащие все слова из запроса пользователя, независимо от их порядка или местоположения в тексте подсказки. Пример: на запрос «смотреть» выдаются все подсказки, где встречается это слово.
- 2. Поиск по префиксам: Поиск должен работать на лету, в процессе набора, находя подсказки по начальным частям слов (префиксам). Пример: при вводе «смотр» должны находиться те же подсказки, что и для полного слова «смотреть».
- 3. Масштабируемость и скорость: Система должна обрабатывать десятки миллионов подсказок и возвращать результат за несколько миллисекунд, чтобы обеспечить мгновенную реакцию на действия пользователя.

Механизм автодополнения на основе префиксов:

- 1. Лексикографическая организация данных. Все подсказки предварительно сортируются в алфавитном порядке по своему тексту. Каждой подсказке присвоен вес, отражающий её популярность.
- 2. Поиск подходящего диапазона. При вводе пользователем префикса (начала запроса) с помощью бинарного поиска находится подмножество подсказок, текст которых начинаются с этого префикса.
- 3. Ранжирование и выдача результата. Найденный блок подсказок пересортировывается по убыванию веса (популярности) и пользователю отображаются самые популярные подсказки из этого списка.

Для оптимизации используют Radix tree с весами в узлах дерева. Radix-tree (сжатое префиксное дерево) — это оптимизированная структура данных для хранения строк, где общие начала слов склеены в один узел, а ветвление происходит только при различиях. Помогает экономить память и ускоряет поиск за счёт объединения узлов (логарифмическое время поиска). [Картинка](#)



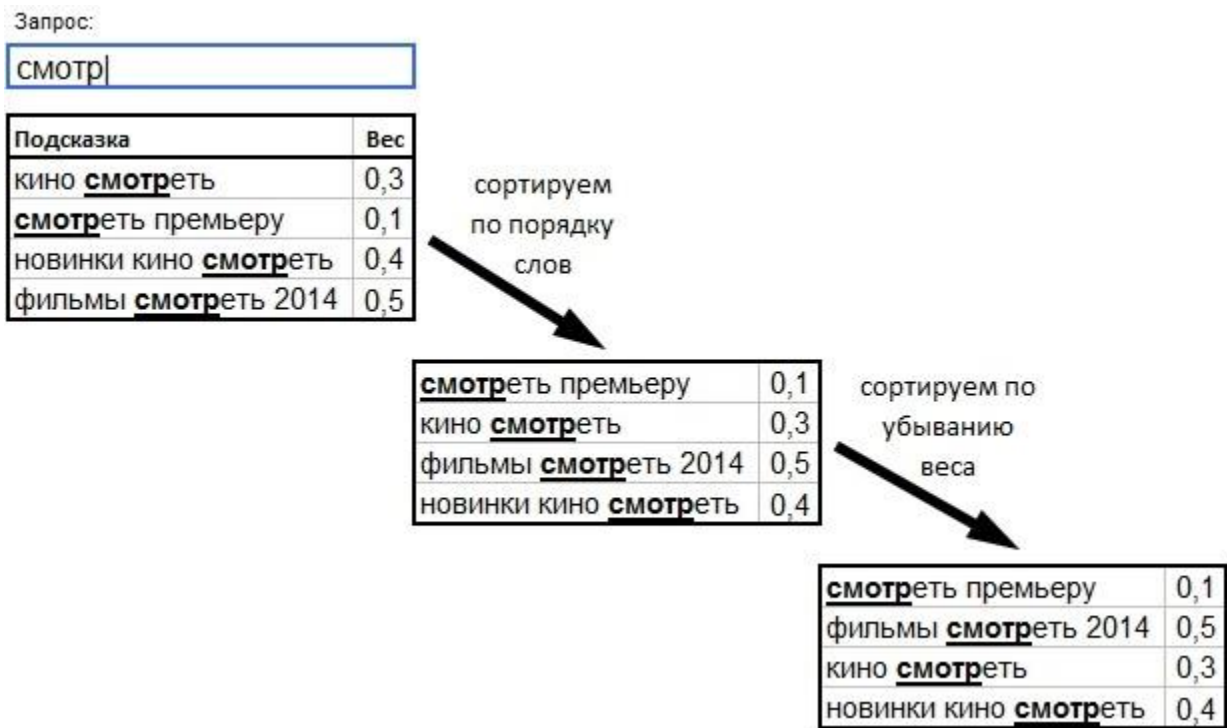
Как работает?

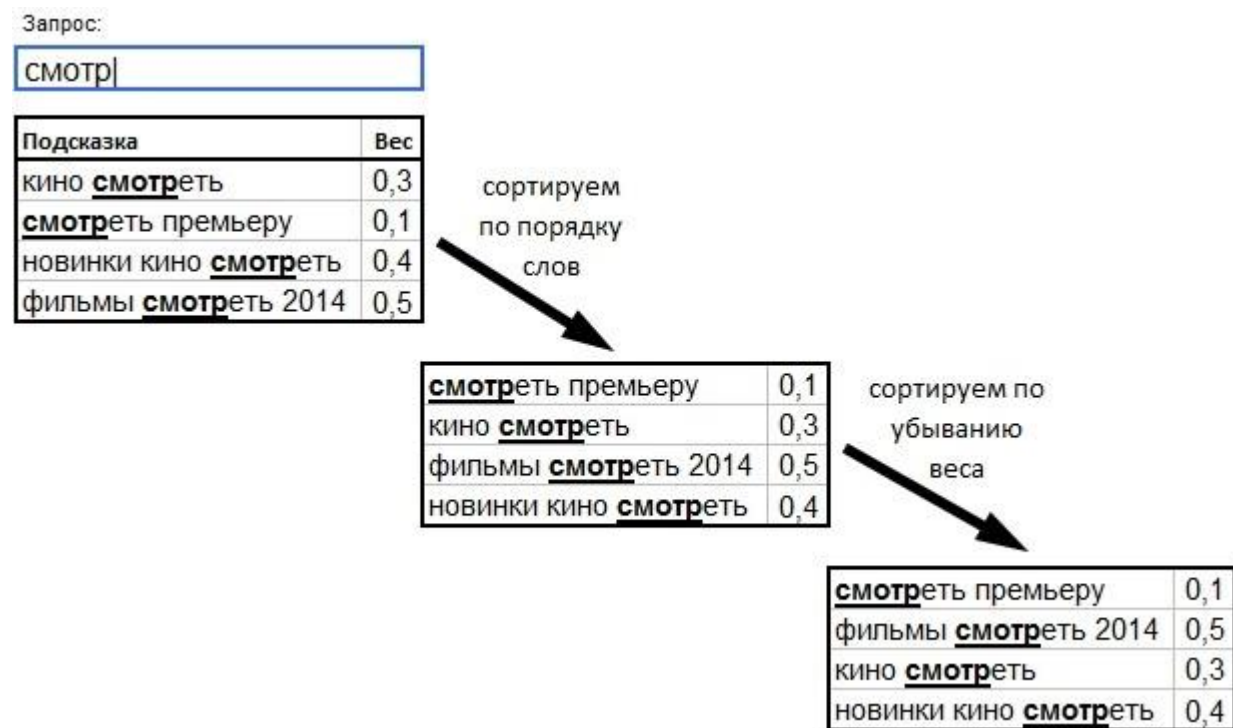
- 1. Каждый узел знает максимум — максимальную популярность среди всех слов в своей ветви.
- 2. Последовательность поиска — сначала проверяем ветви с наибольшим максимумом.
- 3. Отсекаем бесперспективные — если максимум ветви ниже, чем худший из уже найденных результатов, ветвь не исследуем.

Полнотекстовое автодополнение

Алгоритм:

- 1. Токенизация. Запрос пользователя и тексты подсказок разбиваются на отдельные слова (токены).
- 2. Поиск по условию. Для заданного запроса система ищет в базе подмножество подсказок, удовлетворяющих условию: каждое слово из запроса должно совпадать с началом какого-либо слова в подсказке. Позиция этих слов внутри подсказки значения не имеет.
- 3. Ранжирование результатов. Найденное подмножество сортируется по двум последовательным критериям:
 - по убыванию степени соответствия слов подсказки словам запроса. Учитывается близость и порядок найденных слов внутри подсказки.
 - по убыванию предустановленного веса (популярности) подсказки.
- 4. Возврат результата пользователю





Для быстрого нахождения подсказок по словам запроса используются две взаимодополняющие структуры данных:

- прямой индекс;
- обратный индекс.

Прямой и обратный индексы

Прямой индекс — список документов, в котором можно найти этот документ по его id. Иными словами, прямой индекс — это массив строк (вектор документов), где id документа — это его индекс.

Обратный индекс — это словарь, составленный из всех уникальных слов, извлечённых из коллекции документов. Каждому такому слову сопоставлен упорядоченный список идентификаторов документов (posting list), содержащих данное слово.

По этим двум структурам данных достаточно легко найти все документы, удовлетворяющие пользовательскому запросу. Для этого нужно:

1. В обратном индексе: по словам из запроса найти списки id тех документов, где эти слова встречались. Получить пересечение этих списков — результирующий список id документов, где встречаются все слова из запроса.
2. В прямом индексе: по полученным id найти исходные документы и вернуть их пользователю.

Описанные подходы обеспечивают быстрое автодополнение при точном вводе. Но что делать, если пользователь сделал опечатку или орфографическую ошибку? Для этого в базовых алгоритмах применяется нечеткий поиск.

Нечеткий поиск

Основные алгоритмы:

1. **Расстояние Левенштейна.** Расстояние Левенштейна определяет какое минимальное количество односимвольных операций (вставки, удаления, замены) необходимо для превращения одной последовательности в другую.

		0	1	2
	•	М	Е	
0	•	0	1	2
1	М	1	0	1
2	У	2	1	1

- Пустая строка
- ← Удаление
- ↑ Вставка
- ↖ Символы одинаковые, ничего не делаем
- ↗ Замена

Как работает?

Если пользователь вводит запрос с ошибкой, система проверяет похожие слова из базы и для каждого считает расстояние Левенштейна. Далее сортирует слова по возрастанию расстояния и возвращает пользователю результат с наименьшим расстоянием.

2. **N-граммы.** Слово разбивается на подстроки фиксированной длины, ищутся слова с похожими фрагментами. Для запроса находятся все слова, у которых есть хоть один общий фрагмент и сортируются по количеству таких общих фрагментов. Пользователю возвращается слово с наибольшим количеством совпадений.
3. **Поиск по синонимам и онтологиям.** Использование заранее созданных сетей связей между словами для понимания смысла, а не только буквенного состава. Таким образом нечёткий поиск — это механизм, который позволяет системе находить релевантные варианты даже при наличии опечаток, орфографических ошибок или альтернативных написаний в запросе пользователя.

Подводя итог, автодополнение работает так: система в реальном времени ищет в своей базе подсказок варианты, наиболее похожие на уже введенную часть запроса. Для этого она использует префиксные деревья для мгновенного поиска по началу слов и инвертированные индексы для полнотекстового поиска по всем словам. Нечёткие алгоритмы подстраховывают, исправляя опечатки. Найденные варианты сортируются по популярности и релевантности, и топ лучших выводится пользователю — всё это за время между двумя нажатиями клавиш.

Источники: [Поисковые подсказки изнутри Radix tree](#) Более глубокое исследование: [An Introduction to Information Retrieval](#)

Что можно добавить?

Давайте рассмотрим другие структуры данных помимо Radix-tree, которые можно использовать для быстрого автодополнения.

Trie (префиксное дерево)

Базовая структура, от которой произошли все остальные. Каждый символ строки — отдельный узел; путь от корня до листа образует слово.

Плюсы: простота реализации, поиск по префиксу за $O(k)$, где k - длина префикса.

Минусы: катастрофический расход памяти. Для каждого узла хранится массив из 26–33 дочерних указателей (по числу букв алфавита), большинство из которых пусты. При десятках миллионов подсказок это становится неприемлемым. Именно поэтому чистый алгоритм Trie на практике почти не используется - он служит лишь концептуальной основой для более эффективных структур.

DAWG (ориентированный ациклический граф слов)

DAWG превосходит Radix Tree: он сжимает не только общие префиксы, но и общие суффиксы. Одинаковые окончания слов («ание», «ение», «ировать») объединяются в один узел, на который ссылаются несколько путей.

Плюсы: минимально возможный размер для хранения словаря; при работе с большими словарями русского языка экономия памяти по сравнению с Radix Tree может достигать 40–60 %.

Минусы: структура становится направленным графом, а не деревом, что усложняет поиск. Но самое критичное - это то, что DAWG крайне сложно обновлять. Добавление нового слова в уже построенный DAWG может потребовать перестройки значительной части графа. Это делает DAWG непригодным для систем, в которых словарь подсказок обновляется в режиме реального времени. На практике DAWG используют в статических словарях - например, в системах проверки орфографии, где список слов меняется редко.

FST (преобразователь конечных состояний)

FST - наиболее продвинутая структура. Это конечный автомат, который не просто распознает строки, но и сопоставляет каждой строке значение (например, вес подсказки). Он сжимает одновременно и префиксы, и суффиксы, как DAWG, но дополнительно позволяет сохранять произвольные выходные данные на переходах.

Плюсы: минимальный размер среди всех рассмотренных структур при хранении пар (строка → значение); именно FST используется в Apache Lucene - поисковом движке, лежащем в основе Elasticsearch.

Минусы: самая сложная реализация из всех; алгоритм построения минимального конечного автомата нетривиален. Как и DAWG, плохо поддается

динамическому обновлению - как правило, конечный автомат строится заново при каждом обновлении словаря.

Почему Radix Tree побеждает на практике

Несмотря на то, что DAWG и FST теоретически более эффективны с точки зрения использования памяти, Radix Tree остается стандартом для систем автодополнения по целому ряду причин. Во-первых, словарь поисковых подсказок постоянно меняется: появляются новые запросы, обновляются веса подсказок. Radix Tree поддерживает это изначально, в то время как FST и DAWG требуют дорогостоящей перестройки. Во-вторых, хранение максимального веса поддерева в каждом узле - метод, органично встроенный именно в дерево, - позволяет находить топ-N подсказок с эффективным отсеиванием, что крайне важно при миллионах вариантов. В-третьих, выигрыш FST в плане экономии памяти на практике нивелируется кэшированием: «горячие» префиксы (те, которые пользователи вводят чаще всего) все равно хранятся в оперативной памяти отдел.

Хранение и обновление весов в Radix Tree в реальном времени

Разберем два наиболее популярных решения

Что именно хранится в узле?

Каждый узел Radix Tree хранит два принципиально разных значения:

Собственный вес - популярность конкретной подсказки, которая заканчивается в данном узле. Есть только у листьев и у внутренних узлов, соответствующих полному слову.

Максимум поддерева - наибольший собственный вес среди всех подсказок в поддереве данного узла. Есть у каждого узла без исключения. Именно это значение позволяет отсекать бесперспективные ветви при поиске топ-N.

Проблема в том, что при изменении собственного веса любого узла нужно пересчитать max_weight у всех его предков вплоть до корня. Это и есть главная сложность.

Решение 1. Отложенное обновление (Lazy Propagation)

Самый распространённый подход на практике. Веса обновляются не мгновенно, а периодически - батчами.

Как работает:

Поток запросов пользователей

↓

Счётчик кликов (in-memory, атомарный)

↓ (раз в N секунд / M запросов)

Агрегатор пересчитывает веса

↓

Обновление дерева (swap на новую версию)

Каждый пользовательский запрос атомарно инкрементирует счётчик в простой хеш-таблице {подсказка → счётчик}. Это дешево и не касается дерева.

Фоновый процесс периодически читает счётчики, пересчитывает веса и строит новую версию дерева (или обновляет существующую). Старая версия продолжает обслуживать запросы до завершения обновления.

Плюсы: чтение никогда не блокируется; обновление простое и предсказуемое.

Минусы: веса отстают от реальности на время одного окна (обычно от 10 секунд до нескольких минут). Для большинства систем это приемлемо - популярность меняется медленно.

Решение 2. Скользящее окно популярности

Наивный счётчик накапливает клики за всё время существования подсказки. Это создаёт проблему: запрос, который был популярен три года назад, но устарел сегодня, будет занимать высокие позиции вечно.

Решение — считать не абсолютную популярность, а взвешенную по времени.

Экспоненциальное затухание:

$\text{новый_вес} = \text{старый_вес} \times \alpha + \text{текущие_клики} \times (1 - \alpha)$

Где α - коэффициент затухания (например, 0.9). Каждое обновление «забывает» часть прошлого. Запросы недельной давности влияют на вес значительно меньше, чем вчерашние.

Дискретное скользящее окно:

Хранить клики отдельно по слотам времени:

[слот_сегодня: 1500, слот_вчера: 1200, слот_3дня: 800, ...]

Вес = сумма слотов с убывающими коэффициентами

Это позволяет улавливать сезонность (новогодние запросы) и быстро реагировать на вирусные события.

Абсолютное большинство крупных систем используют Lazy Propagation в сочетании со скользящим окном - это даёт достаточную актуальность весов при минимальной инженерной сложности. Задержка в несколько минут между реальным ростом популярности запроса и его отражением в подсказках незаметна для пользователя.

Улучшенные алгоритмы нечеткого поиска

Расстояние Дамерау–Левенштейна vs классическое

Классическое расстояние Левенштейна:

Допускает три операции над символами:

- **Вставка:** «кино» → «киное»
- **Удаление:** «кино» → «кно»
- **Замена:** «кино» → «цино»

Каждая операция стоит 1. Расстояние — минимальная суммарная стоимость преобразования одной строки в другую.

Проблема: самая частая опечатка не покрывается

Исследования раскладок клавиатуры показывают, что около 80% опечаток при наборе - это транспозиция: два соседних символа меняются местами. Пальцы нажимают правильные клавиши, но в неправильном порядке.

«фильм» → «фильм» (правильно)

«фильм» → транспозиция «ь» и «м»

«фильмы» → «фильмы» (правильно)

«лифъм» → две транспозиции

Классическое расстояние Левенштейна не имеет операции транспозиции. Для него «фильм» → «фильм» стоит 2(удалить «м», вставить «м» после «ь»), хотя интуитивно это одна опечатка.

Расстояние Дамерау–Левенштейна добавляет четвёртую операцию:

Транспозиция: «фильм» → «фильм» — стоит 1

ВК-деревья

ВК-дерево - это структура данных для быстрого поиска похожих слов в большом словаре. Вместо того чтобы сравнивать запрос с каждым словом по очереди, оно организует словарь так, что большинство слов можно пропустить без проверки.

В основе лежит простое свойство расстояния Левенштейна: если слово А далеко от слова В, а В близко к запросу, то А не может быть близко к запросу. Это позволяет при обходе дерева целыми ветвями отсекал слова, которые заведомо не подойдут - не вычисляя для них расстояние вообще.

В результате при поиске с допуском на одну ошибку алгоритм проверяет лишь 5–15% словаря вместо 100%, что и обеспечивает его практическую эффективность.

Порог расстояния

Допустимое расстояние при нечётком поиске не может быть фиксированным для всех слов: расстояние 1 для двухбуквенного слова означает полную замену половины строки, тогда как для длинного слова - незначительную погрешность. Стандартный подход, применяемый в большинстве промышленных систем:

- длина 1–2 символа - порог 0 (только точное совпадение);
- длина 3–5 символов - порог 1 (одна ошибка);
- длина 6–8 символов - порог 1;
- длина 9 и более символов - порог 2 (две ошибки).

Порог выше 2 на практике не используется: при большем допуске короткое слово может совпасть с совершенно несвязанным, что разрушает качество результатов. Кроме того, однобуквенные и двухбуквенные токены внутри составного запроса нечёткому поиску не подвергаются - предлог «в» с опечаткой «б» является другим запросом, а не ошибкой.

Подводя итог, автодополнение работает так: система в реальном времени ищет в своей базе подсказок варианты, наиболее похожие на уже введённую часть запроса. Для этого она использует префиксные деревья для мгновенного поиска по началу слов и инвертированные индексы для полнотекстового поиска по всем словам. Нечёткие алгоритмы - расстояние Дамерау–Левенштейна и ВК-деревья - подстраховывают систему, исправляя опечатки без перебора всего

словаря. Найденные варианты сортируются по популярности и релевантности, а веса подсказок непрерывно обновляются, отражая актуальные пользовательские предпочтения. Весь этот процесс укладывается в промежуток между двумя нажатиями клавиш.

